

23PCPE402 SENSORS AND ACTUATORS

Project Based Learning (PBL)

AY 2025-26

“VISUAL COLOUR DETECTION”

Prepared By:

Samruddhi Anuse	- UEC2024206
Shrushti Bidkar	- UEC2024209
Vaishnavi Deshmukh	- UEC2024214
Nikita Thorat	- UEC2024262
Dipika Vaishnav	- UEC2024264

Guided By:

Prof: S. L. Sahare/Prof. S. G. Dube

1.Introduction to your project

The Visual Colour Detection System with Voice Output is an embedded system project that detects the color of an object using a color sensor and displays the detected color on a computer screen using Processing software.

In addition to visual output, the system also provides audio feedback by pronouncing the detected color through the computer speakers using text-to-speech technology.

The system is built using an Arduino microcontroller and a color sensor (TCS3200). The sensor detects RGB values, which are processed by Arduino to identify the color. The detected color data is sent to the Processing software, which displays the color visually and announces it using voice output.

This project is useful in automation, assistive technology for visually impaired people, and smart systems

2. Aim of PBL

To design and implement a smart system using Arduino and a color sensor that detects colors in real-time and provides both visual display and voice output using Processing software.

3. General Block Diagram

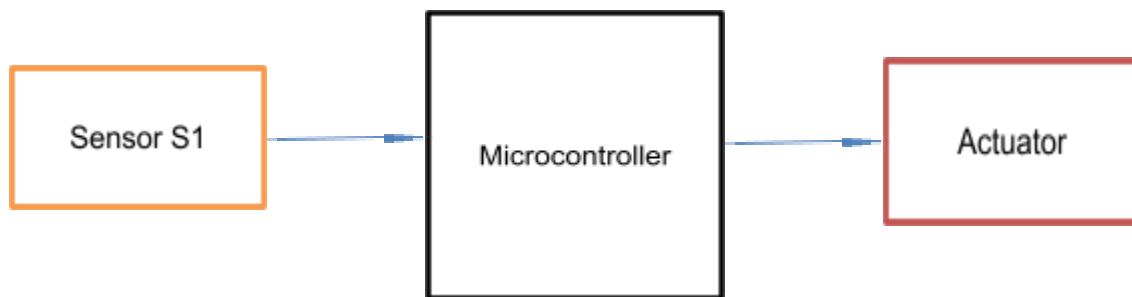


Figure 1 Block Diagram of System

4. Explanation of System

- The system detects the color of an object placed in front of the sensor by analyzing the light reflected from the object's surface.
- When light falls on the object, it reflects different amounts of Red, Green, and Blue components depending on its color, which are captured by the color sensor.
- The color sensor measures these RGB components and converts the light intensity into corresponding frequency signals for each color.
- The Arduino reads these raw frequency values from the sensor using its input pin and processes them for further analysis.
- These raw frequency values are then converted into standard RGB intensity values ranging from 0 to 255 using a mapping function.
- After conversion, a color detection algorithm compares the RGB values and determines which color is dominant.
- Based on predefined conditions, the system identifies the color name such as Red, Green, Blue, Yellow, etc.
- Once the color is identified, the Arduino sends both the RGB values and the detected color name to the computer through serial communication.
- The Processing software running on the computer receives this data and separates the RGB values and color name.
- The software then displays the detected color visually by changing the background color of the screen according to the RGB values.
- Along with the visual representation, the name of the detected color is also displayed clearly on the screen for user understanding.

5. Selection of Components

1. Arduino UNO

- Used as the main controller
- Processes sensor data

2. Color Sensor Module (TCS3200)

- Detects RGB values
- Has built-in LED for illumination

3. Jumper Wires

- Used to connect sensor to Arduino

4. USB Cable

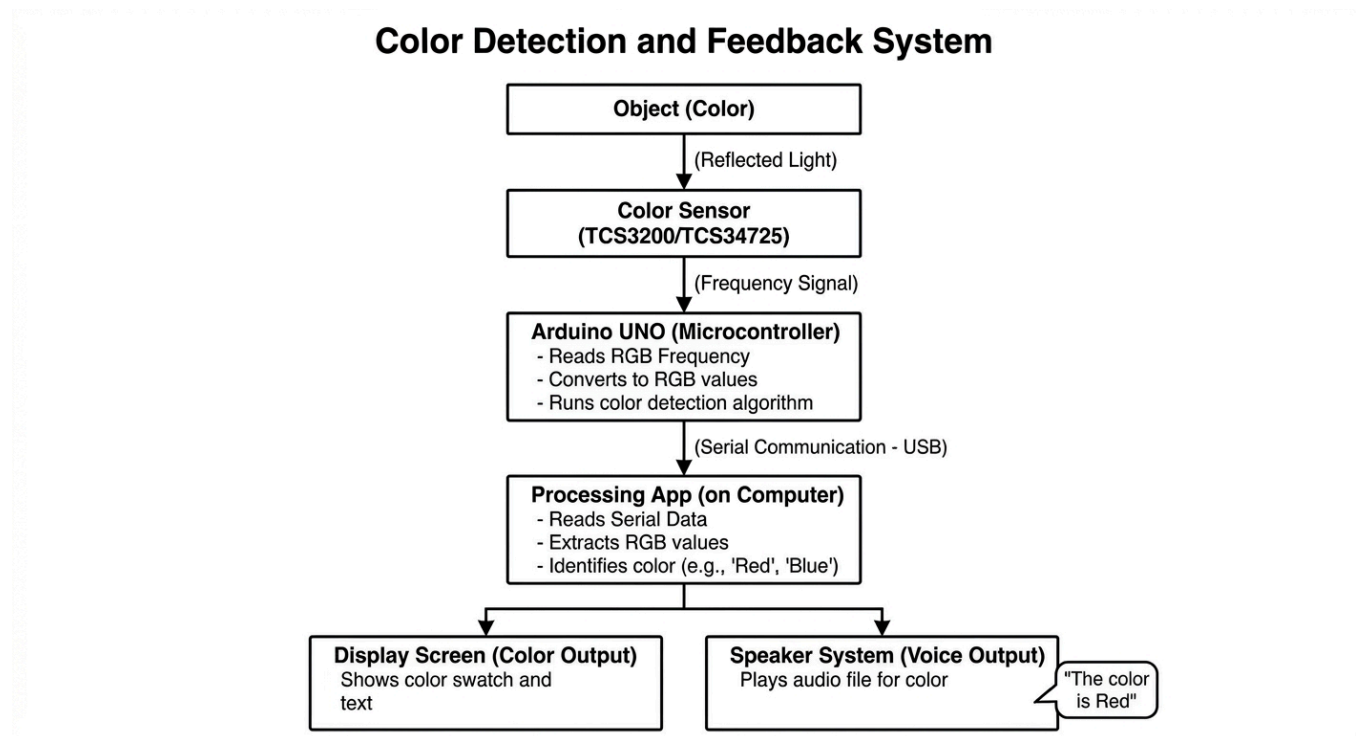
- Provides power to Arduino
- Enables serial communication

5. Laptop / Computer with Processing Software

- Displays detected color
- Provides voice output

6. Interfacing Sensors and Actuators with Microcontrollers

Block Diagram



7. Explanation

1. Input Stage

- Object: Reflects light to the sensor for color analysis.

2. Color Sensor (TCS3200)

- Function: Converts reflected light intensity into frequency signals via RGB filters.
- Pin Connections:
 - S0 & S1 (Pins 4, 5): Frequency scaling control.
 - S2 & S3 (Pins 6, 7): Color filter selection (Red, Green, Blue).
 - OUT (Pin 8): Sends frequency signal to Arduino.
 - Power: VCC to 5V and GND to Ground.
- Logic: Selects filters (LOW/HIGH combinations) and converts light intensity into a frequency signal.

3. Arduino UNO (Processing)

- Input: Receives frequency from Pin 8.
- Port Logic: Digital Pins 4–7 act as outputs to configure the sensor
- Processing: Maps frequency to RGB values (0-255) and identifies the color name.
- Communication: Sends data via USB Serial at 9600 baud rate.

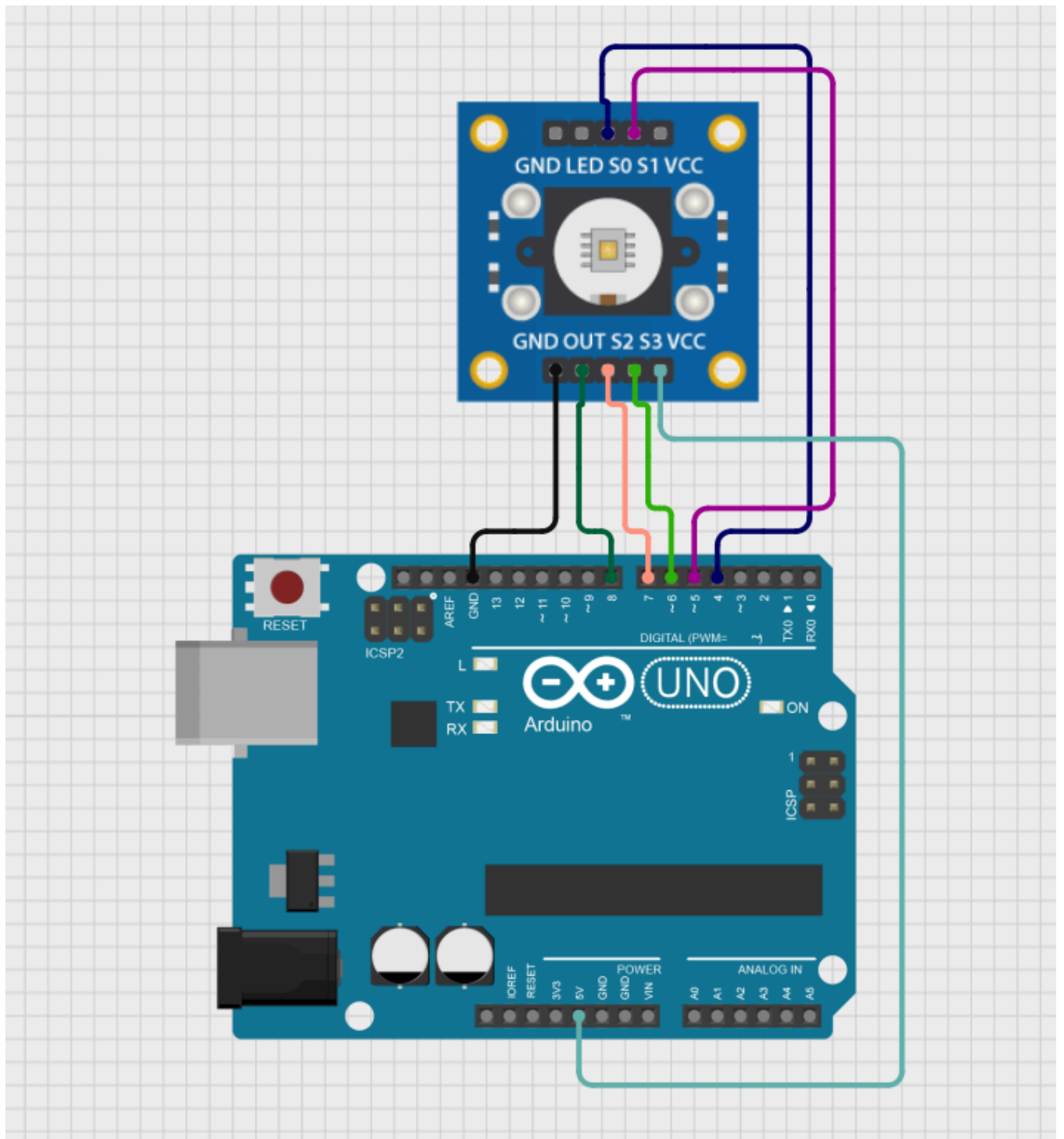
4. Software & Output

- Serial Format: Transmits data as R,G,B ColorName.
- Processing Software: * Extracts RGB values from the serial stream.
 - Changes the screen background to the detected color.
 - Displays the color name text.
- Voice Output: Uses Text-to-Speech (TTS) to announce the color name whenever a change is detected.

5. System Flow Summary

1. Sense: Sensor captures light from the object.
2. Process: Arduino converts frequency to RGB and identifies the color.
3. Transfer: Data is sent to the PC via USB.
4. Output: Processing software displays the color and speaks the name.

8. Circuit Diagram



9. Explanation

The circuit consists of an Arduino UNO interfaced with a TCS3200 Color Sensor and connected to a computer running Processing IDE.

Color Sensor (TCS3200):

- VCC → 5V of Arduino
- GND → GND
- S0, S1 → Pins 4 and 5 (frequency scaling)
- S2, S3 → Pins 6 and 7 (color selection)
- OUT → Pin 8 (frequency output)

The sensor detects the color of an object and converts light intensity into frequency signals.

Arduino UNO:

- Reads frequency from sensor
- Converts it into RGB values
- Identifies the color using programmed conditions
- Sends RGB values and color name to the computer via USB

Processing Software & Output:

- Receives data from Arduino
- Displays the detected color on screen

- Shows the color name
- Uses text-to-speech to announce the color through speakers

Power Supply:

- The system is powered using the Arduino USB connection
- No external components like LEDs or buzzers are required

Working

1. Color Sensing:

The sensor detects RGB components of the object by selecting filters using S2 and S3 pins.

2. Signal Processing:

Arduino reads frequency values using pulseIn() and converts them into RGB values (0–255).

3. Color Detection:

RGB values are compared to identify the color (Red, Green, Blue, Yellow, etc.).

4. Data Transmission:

The detected RGB values and color name are sent to Processing software via serial communication.

5. Output Display:

Processing:

- Changes screen color
- Displays the detected color name

6. Voice Output:

When a new color is detected, Processing uses **Microsoft PowerShell** to speak the color name through the computer speaker.

7. Continuous Operation:

The system continuously detects and updates color in real-time.

10. Code

10.1 Purpose of the Arduino Code:

The code uses a TCS3200 color sensor with an Arduino to measure the RGB values of an object by analyzing reflected light frequencies. It processes and averages the sensor readings to reduce noise, converts them into calibrated RGB values, and identifies the object's color based on predefined thresholds. When a new color is detected, the system sends the RGB values and corresponding color name via serial communication for visualization or voice feedback.

Code Breakdown:

// Define the pins for the TCS3200 color sensor

#define S0 4 // Frequency scaling pin

#define S1 5 // Frequency scaling pin

#define S2 6 // Photodiode selection pin

#define S3 7 // Photodiode selection pin

#define sensorOut 8 // Output pin from sensor

int red, green, blue; // Variables to store mapped RGB values

int rRaw, gRaw, bRaw; // Variables to store raw frequency values

String colorName = ""; // Stores detected color name

String lastColor = ""; // Stores previous color to avoid repetition

// Function to detect color based on RGB values

String detectColor(int r, int g, int b) {

// BLACK / WHITE / GRAY detection

if (r < 40 && g < 40 && b < 40) return "BLACK";

if (r > 200 && g > 200 && b > 200) return "WHITE";

if (abs(r - g) < 15 && abs(g - b) < 15) return "GRAY";

// PRIMARY COLORS

if (r > g && r > b && g < 100 && b < 100) return "RED";

```
if (g > r && g > b && r < 100 && b < 100) return "GREEN";
```

```
if (b > r && b > g && r < 100 && g < 100) return "BLUE";
```

```
// SECONDARY COLORS
```

```
if (r > 180 && g > 180 && b < 100) return "YELLOW";
```

```
if (g > 180 && b > 180 && r < 100) return "CYAN";
```

```
if (r > 180 && b > 180 && g < 100) return "MAGENTA";
```

```
// EXTRA COLORS
```

```
if (r > 200 && g > 120 && b < 80) return "ORANGE";
```

```
if (r > 200 && g > 150 && b > 150) return "PINK";
```

```
if (r > 100 && g < 80 && b > 100) return "PURPLE";
```

```
if (r > 120 && g < 80 && b < 50) return "BROWN";
```

```
return "UNKNOWN"; // If no match found
```

```
}
```

```
// Function to read color with averaging (noise reduction)
```

```
int readColor(bool s2, bool s3) {
```

```
    digitalWrite(S2, s2); // Select photodiode filter
```

```
    digitalWrite(S3, s3);
```

```
    int sum = 0;
```

```
// Take 5 readings to reduce noise
```

```
    for (int i = 0; i < 5; i++) {
```

```
        sum += pulseIn(sensorOut, LOW); // Read frequency
```

```
    }
```

```

return sum / 5; // Return average value

}

void setup() {

    Serial.begin(9600); // Start serial communication

    // Set pin modes

    pinMode(S0, OUTPUT);

    pinMode(S1, OUTPUT);

    pinMode(S2, OUTPUT);

    pinMode(S3, OUTPUT);

    pinMode(sensorOut, INPUT);


    // Set frequency scaling to 20%

    digitalWrite(S0, HIGH);

    digitalWrite(S1, LOW);

}

void loop() {

    // Read RED component

    rRaw = readColor(LOW, LOW);

    // Read GREEN component

    gRaw = readColor(HIGH, HIGH);

    // Read BLUE component

    bRaw = readColor(LOW, HIGH);

```

```

// Convert raw values to RGB (0–255 range)

red  = map(rRaw, 25, 72, 255, 0);

green = map(gRaw, 25, 72, 255, 0);

blue  = map(bRaw, 25, 72, 255, 0);

// Limit values within valid range

red  = constrain(red, 0, 255);

green = constrain(green, 0, 255);

blue  = constrain(blue, 0, 255);

// Detect color using RGB values

colorName = detectColor(red, green, blue);


// Send data only if color changes

if (colorName != lastColor) {

  Serial.print(red);    // Send RED value

  Serial.print(",");

  Serial.print(green);  // Send GREEN value

  Serial.print(",");

  Serial.print(blue);   // Send BLUE value

  Serial.print(",");

  Serial.println(colorName); // Send detected color name

  lastColor = colorName; // Update previous color

} delay(1000); // Wait 1000 ms before next reading

}

```

10.2 Explanation of Code

1. Pin Definitions and Initialization

The code defines pins (S0–S3 and sensorOut) for interfacing the TCS3200 color sensor with the Arduino. Variables are initialized to store raw frequency values and processed RGB values. In the `setup()` function, all pins are configured, serial communication is started, and frequency scaling is set to 20% for stable sensor operation.

2. Color Measurement (loop() function)

The sensor reads color by selecting different photodiode filters using S2 and S3. The function `readColor()` captures multiple pulse readings for each color (Red, Green, Blue) and averages them to reduce noise. These raw values are then converted into standard RGB values (0–255) using mapping.

3. Colour Detection and Output

The processed RGB values are passed to the `detectColor()` function, which identifies the closest matching color based on predefined thresholds. When a new color is detected (different from the previous one), the RGB values and color name are sent via serial communication for display or further processing.

4. Delay for Stability

A delay of 200 ms is used to control the reading speed, allowing the sensor to stabilize between measurements and reducing rapid fluctuations in detected color.

10.3 Purpose of the Processing Code:

The code establishes serial communication between Arduino and Processing to receive RGB values and detected color names from a TCS3200 sensor, displays the corresponding color visually on the screen, and provides voice feedback using text-to-speech when a new color is detected, with a controlled delay to prevent rapid or repeated announcements.

Code Breakdown:

```
// Import serial library for communication with Arduino
import processing.serial.*;
Serial myPort; // Serial object for communication
```

```
int red = 0, green = 0, blue = 0; // Variables to store RGB values
String colorName = "NONE";      // Variable to store detected color name
String lastColor = "";          // Stores previous color to avoid repetition
long lastSpeakTime = 0;         // Variable to control delay between voice outputs
```

```
void setup() {
// Create display window
size(500, 400);
// Print available COM ports (for selection)
println("Available Ports:");
printArray(Serial.list());
// Initialize serial communication with Arduino
myPort = new Serial(this, Serial.list()[2], 9600); //[2] because COM5 is used.
// Clear old data and wait for complete line
myPort.clear();
myPort.bufferUntil('\n');
}
```

```
void draw() {
// Set background color based on detected RGB
background(red, green, blue);
```

```
// Draw top bar for text visibility
fill(0, 150);
noStroke();
rect(0, 0, width, 90);
// Display title
fill(255);
textAlign(CENTER, CENTER);
textSize(26);
text("Detected Color", width/2, 30);
// Display detected color name
textSize(30);
text(colorName, width/2, 65);
// Display RGB values
fill(0);
textSize(18);
text("R: " + red + " G: " + green + " B: " + blue, width/2, height - 20);
}
```

```
void serialEvent(Serial myPort) {
```



```

// Read incoming serial data
String data = myPort.readStringUntil('\n');
if (data == null) return; // Ignore null data
// Clean and validate data
data = trim(data);
if (data.length() == 0) return;
// Split data into values (R,G,B,Color)
String[] values = split(data, ',');
// Ensure correct format
if (values.length != 4) return;

try {

// Convert received values into integers
int r = Integer.parseInt(values[0].trim());
int g = Integer.parseInt(values[1].trim());
int b = Integer.parseInt(values[2].trim());
String cname = values[3].trim();

// Validate RGB range
if (r < 0 || r > 255 || g < 0 || g > 255 || b < 0 || b > 255) return;
// Update values
red = r;

green = g;
blue = b;
colorName = cname;
// Speak only if color changes and 1 second delay passed
if (!colorName.equals(lastColor) && millis() - lastSpeakTime > 1000) {
println(colorName); // Print detected color
speakColor(colorName); // Call voice function
lastColor = colorName; // Update previous color
lastSpeakTime = millis(); // Update time
}
}
catch (Exception e) {
// Handle invalid data
println("Invalid data received");
}
}

```

// TEXT TO SPEECH FUNCTION

```
void speakColor(String text) {  
  try {  
  
    // PowerShell command for voice output  
    String command = "powershell -Command \"Add-Type -AssemblyName System.Speech; \" +  
    \"$speak = New-Object System.Speech.Synthesis.SpeechSynthesizer; \" +  
    \"$speak.Speak('\" + text + '\"');\"";  
    // Execute command  
    Runtime.getRuntime().exec(command);  
  
  } catch (Exception e) {  
  
    println("Speech Error"); // Handle error  
  }  
}
```

10.2 Explanation of Processing Code

5. Serial Initialization and Setup

The code imports the serial library and establishes communication with Arduino using a selected COM port. Variables are initialized to store RGB values and detected color names. In the `setup()` function, the display window is created, available ports are listed, and the serial buffer is configured to read complete lines of incoming data.

6. Color Visualization (draw() function)

The `draw()` function continuously updates the screen background based on the received RGB values, visually representing the detected color. A top overlay is added for clarity, and the detected color name along with RGB values is displayed on the screen.

7. Data Reception and Processing (serialEvent() function)

The system reads incoming serial data from Arduino, trims unwanted characters, and splits it into RGB values and the color name. These values are validated to ensure they are within the correct range before updating the display variables.

8. Color Change Detection and Voice Output

The code checks whether the detected color is different from the previous one.

9. Text-to-Speech Function

A PowerShell command is used to convert the detected color name into speech using the system's speech synthesizer, providing audible feedback to the user.

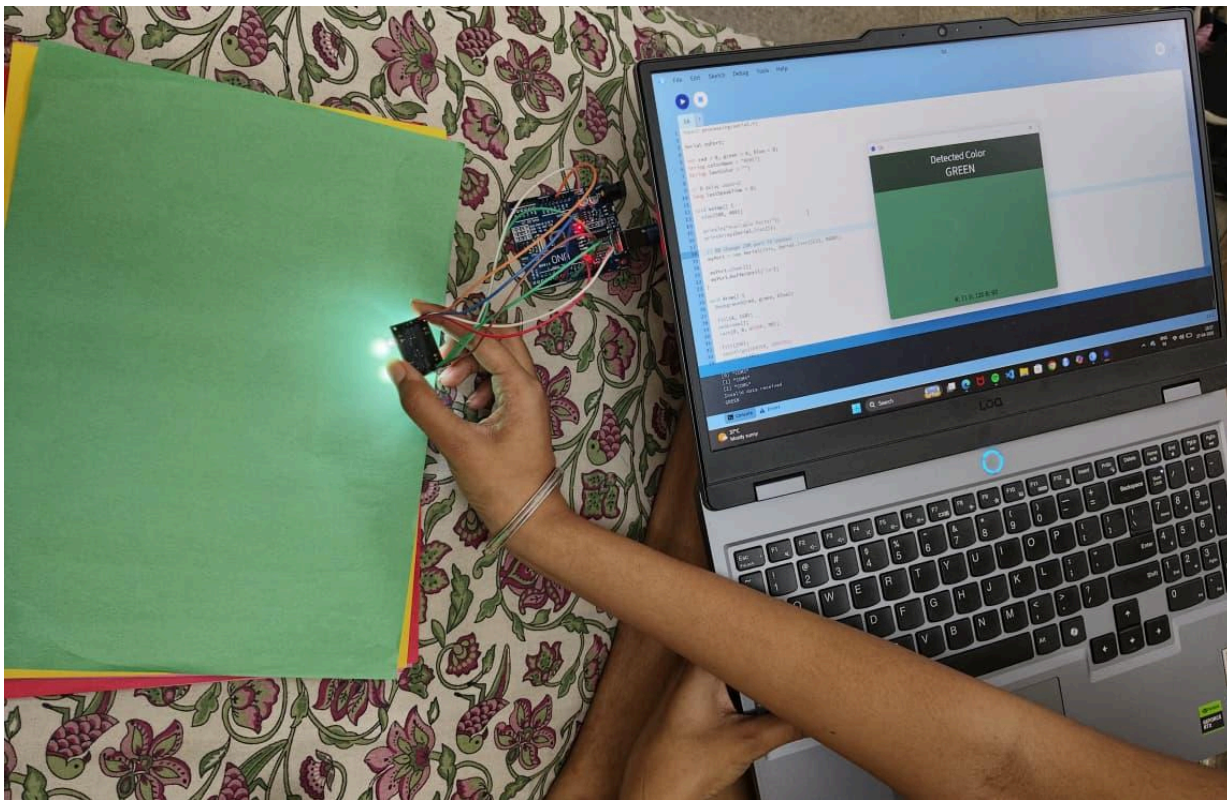
10. Delay Control for Stability

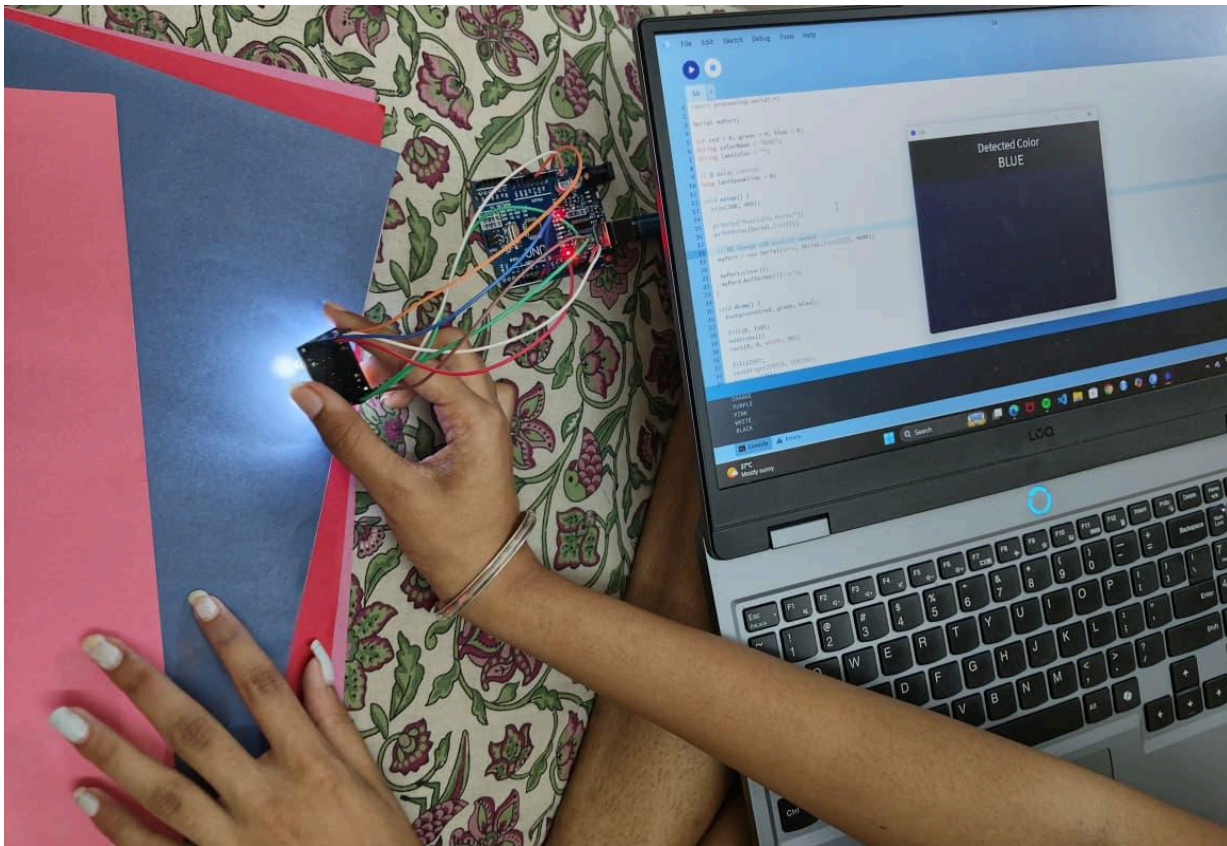
A timing condition using `millis()` ensures a 1-second delay between voice outputs, stabilizing the system and avoiding overlapping or frequent voice triggers due to sensor fluctuations.

Summary

The system continuously reads RGB values from the TCS3200 color sensor, processes them to detect the object's color, and displays it on the screen. When a new color is identified, it provides voice feedback with a controlled delay, ensuring stable and non-repetitive output.

11. Photo





12. Video Link

[Visual Color Detection](#)